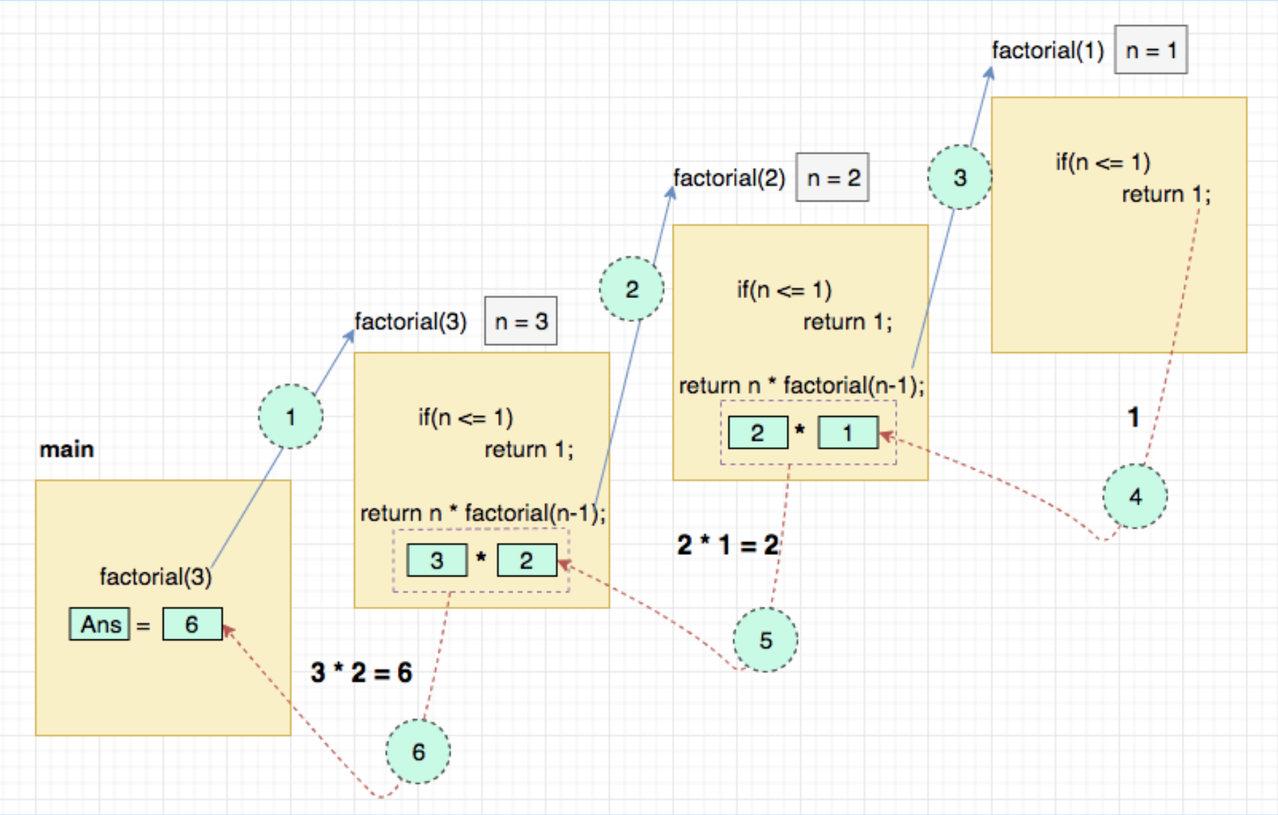
Recursion

SHSID CPC

Basic Concept

We use a function inside itself.
We first define a function. When you write code to define it, you would use the function similar to how you normally use your other functions.



For a recursive function, try to identify three things:

1. Find what the function should do
2. Find when the function should stop using itself
3. Find how and when the function should use itself

Practical Applications

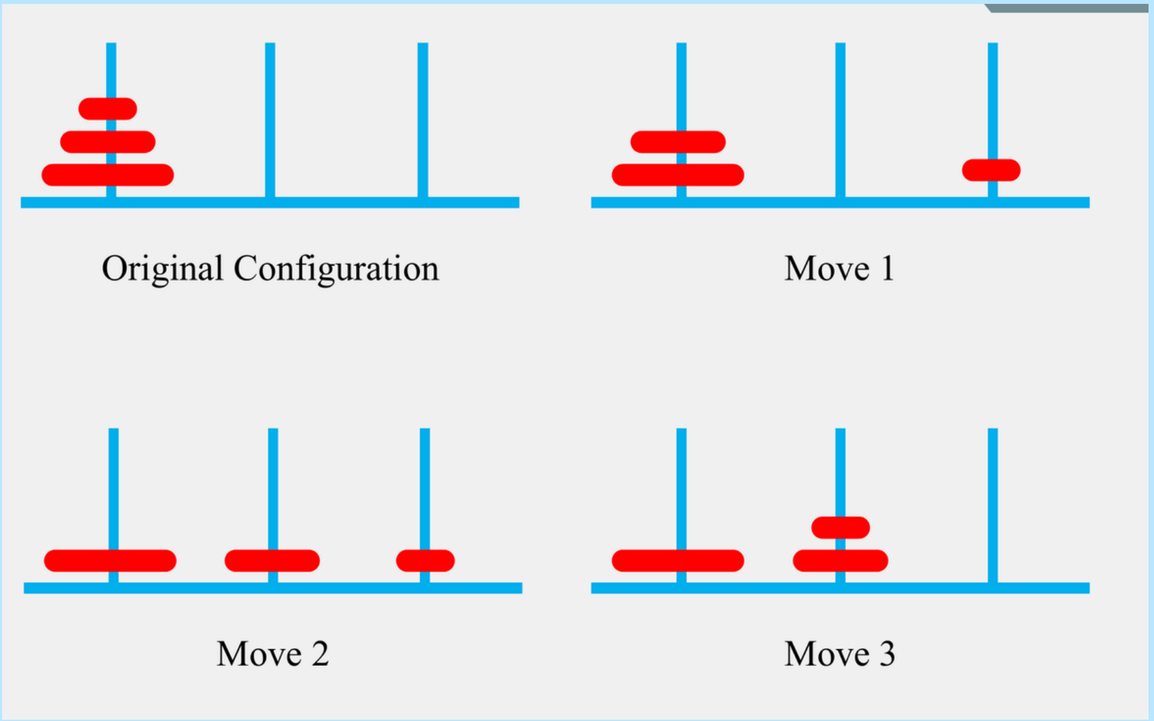
The GCD function!

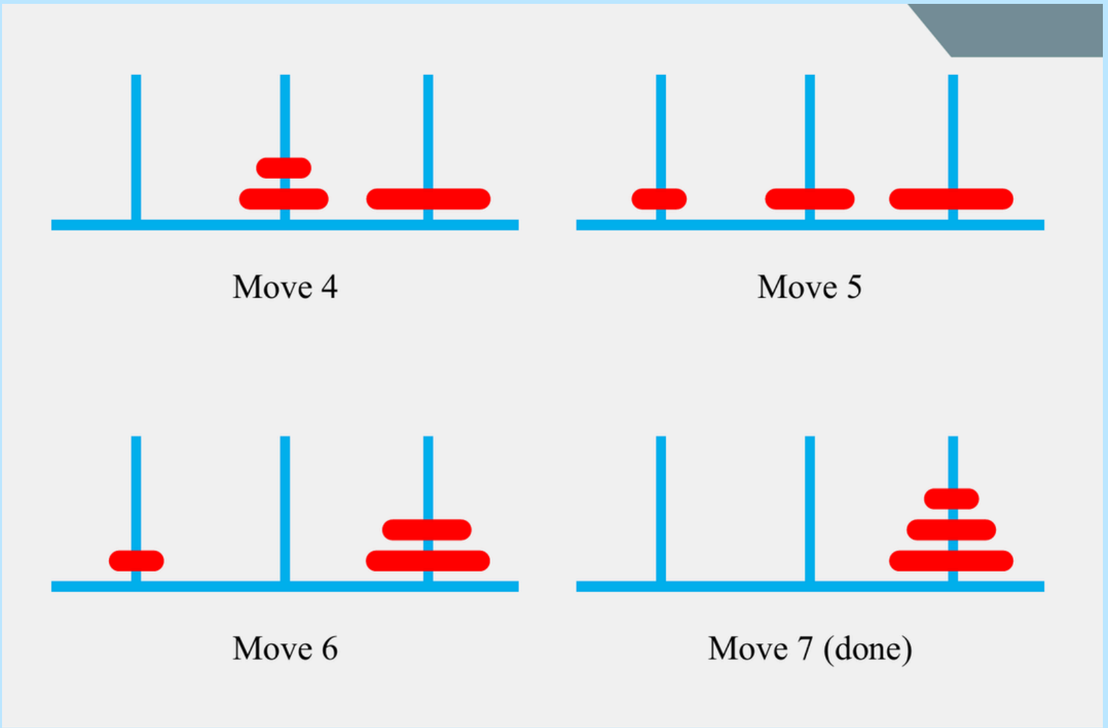
We can use the Euclidean Algorithm to do the job. The Euclidean Algorithms basically tells us to mod the bigger number by the smaller number. If the result of modding is not 0, we do the gcd again with the smaller number and the result of modding. If the result of modding is 0, we return the value of the smaller number as the gcd.

(Sample code shown to the right)

```
def gcd(p,q){
    if(q==0){
        return p
    }
    else{
        return gcd(q,p%q)
    }
}
```

Ex2: Hanoi’ s Game

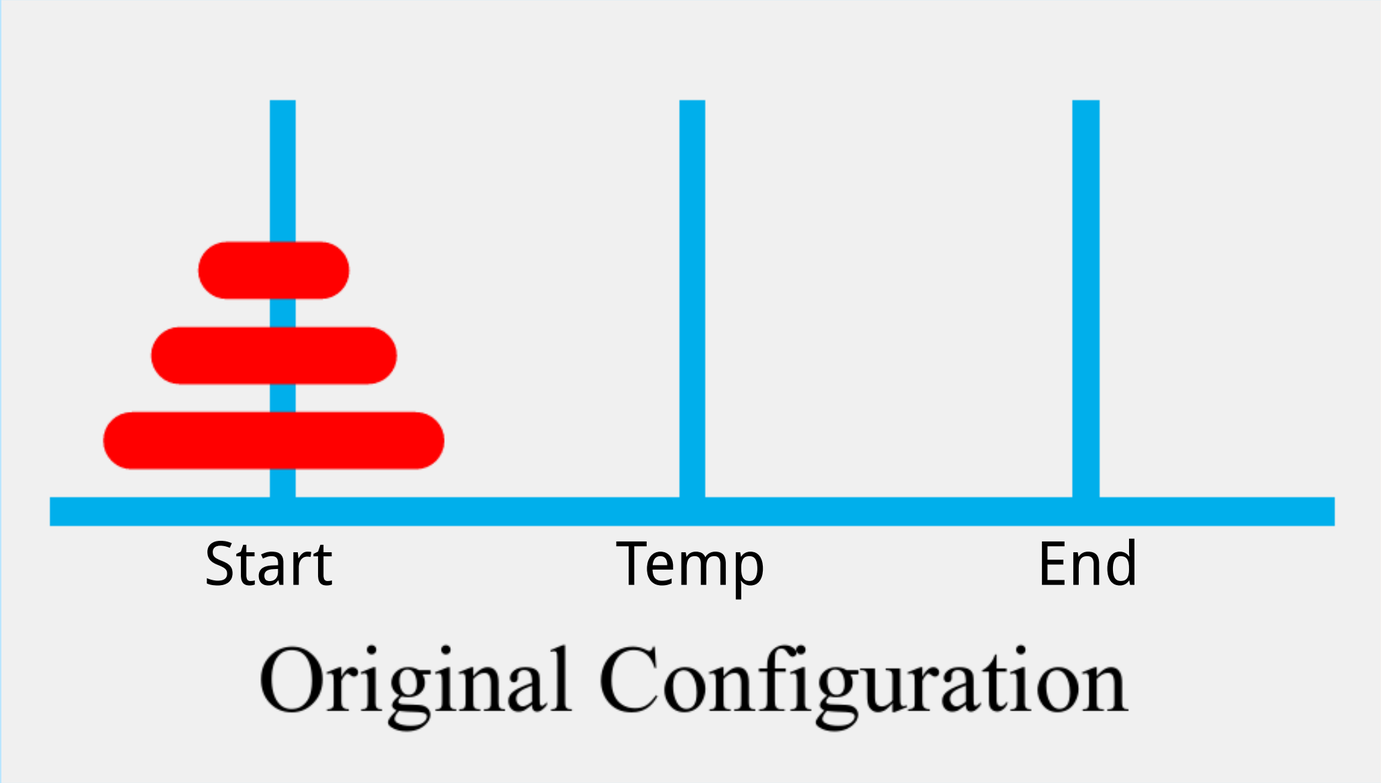




- Move only one disk at a time. You can only take the top disk off a peg.
- No crushing. You can never place a larger disk on top of a smaller disk.

  Find a sequence of instructions to output a way to solve the game

Hanoi’s Game



Hanoi’s Game

Recursive Structure:

- Move N-1 to temp
- Move biggest one to end
- Move N-1 to end

```python
def tower_of_hanoi(n, Start, End, Temp):
    # Base Case
    if n == 1:
        print(f"Move disk 1 from {Start} to {End}")
        return

    # Step 1: moves top N-1 disks from Start to Temp
    tower_of_hanoi(n - 1, Start, Temp, End)

    # Step 2: move the remaining biggest disk to End
    print(f"Move disk {n} from {Start} to {End}")

    # Step 3: moves those N-1 disks from Temp to End
    tower_of_hanoi(n - 1, Temp, End, Start)

tower_of_hanoi(3, 'A', 'C', 'B')
```

Hanoi’s Game

Recursive Structure:

- Move N-1 to temp
- Move biggest one to end
- Move N-1 to end

```
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

70. Climbing Stairs

Easy | Topics | Companies | Hint

You are climbing a staircase. It takes `n` steps to reach the top.

Each time you can either climb `1` or `2` steps. In how many distinct ways can you climb to the top?

Example 1:

```
Input: n = 2
Output: 2
Explanation: There are two ways to climb to the top.
1. 1 step + 1 step
2. 2 steps
```

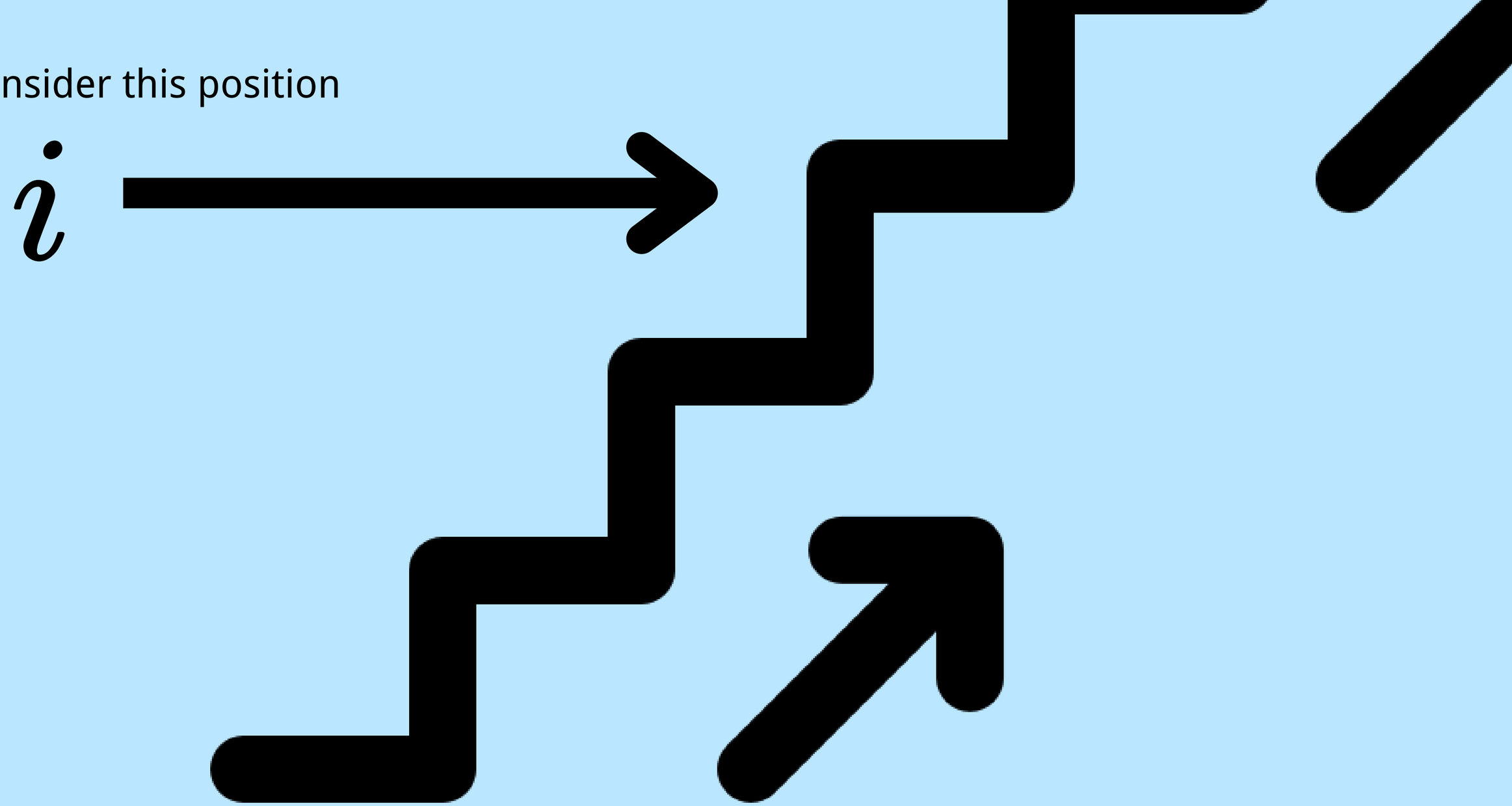
Example 2:

```
Input: n = 3
Output: 3
Explanation: There are three ways to climb to the top.
1. 1 step + 1 step + 1 step
2. 1 step + 2 steps
3. 2 steps + 1 step
```

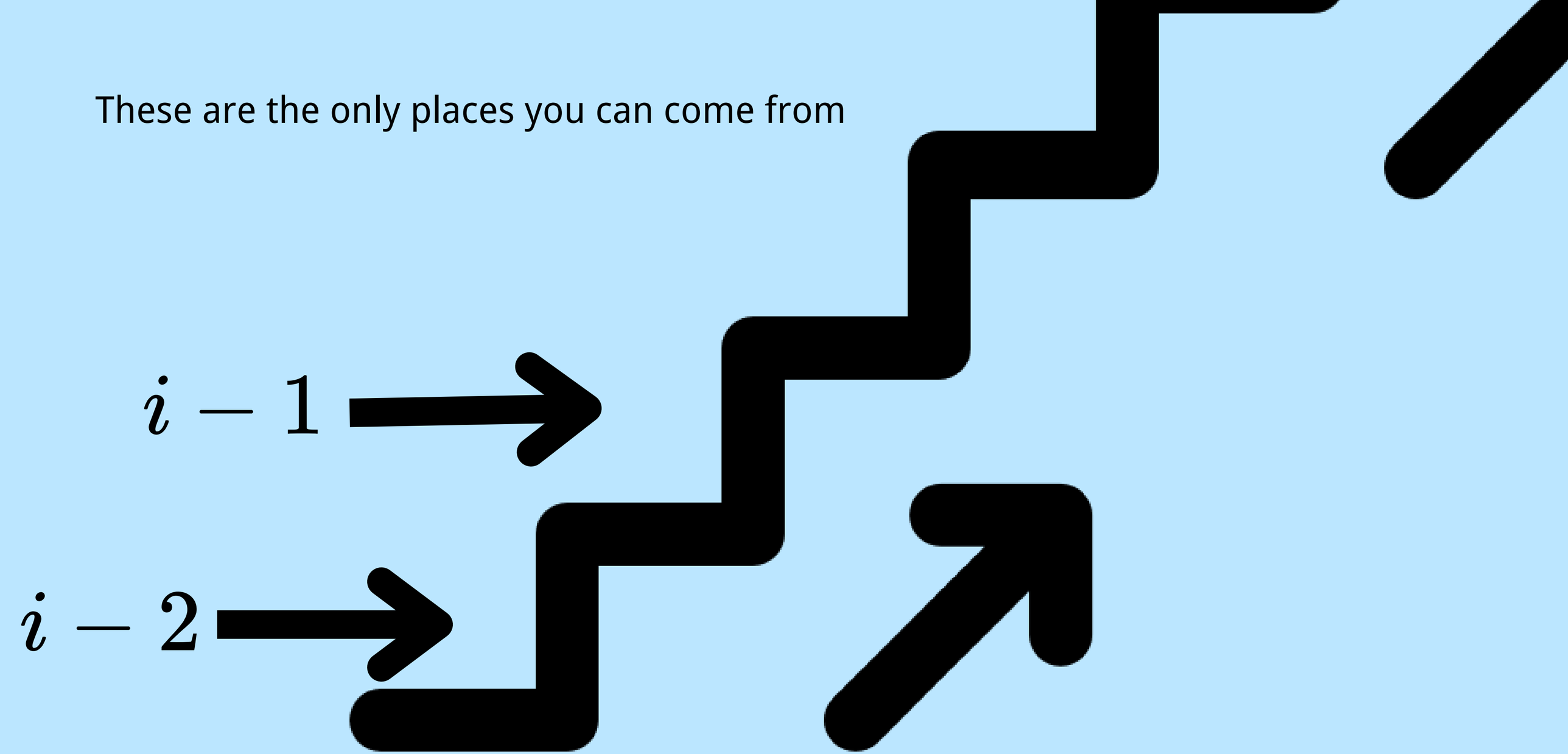
Constraints:

- `1 <= n <= 45`

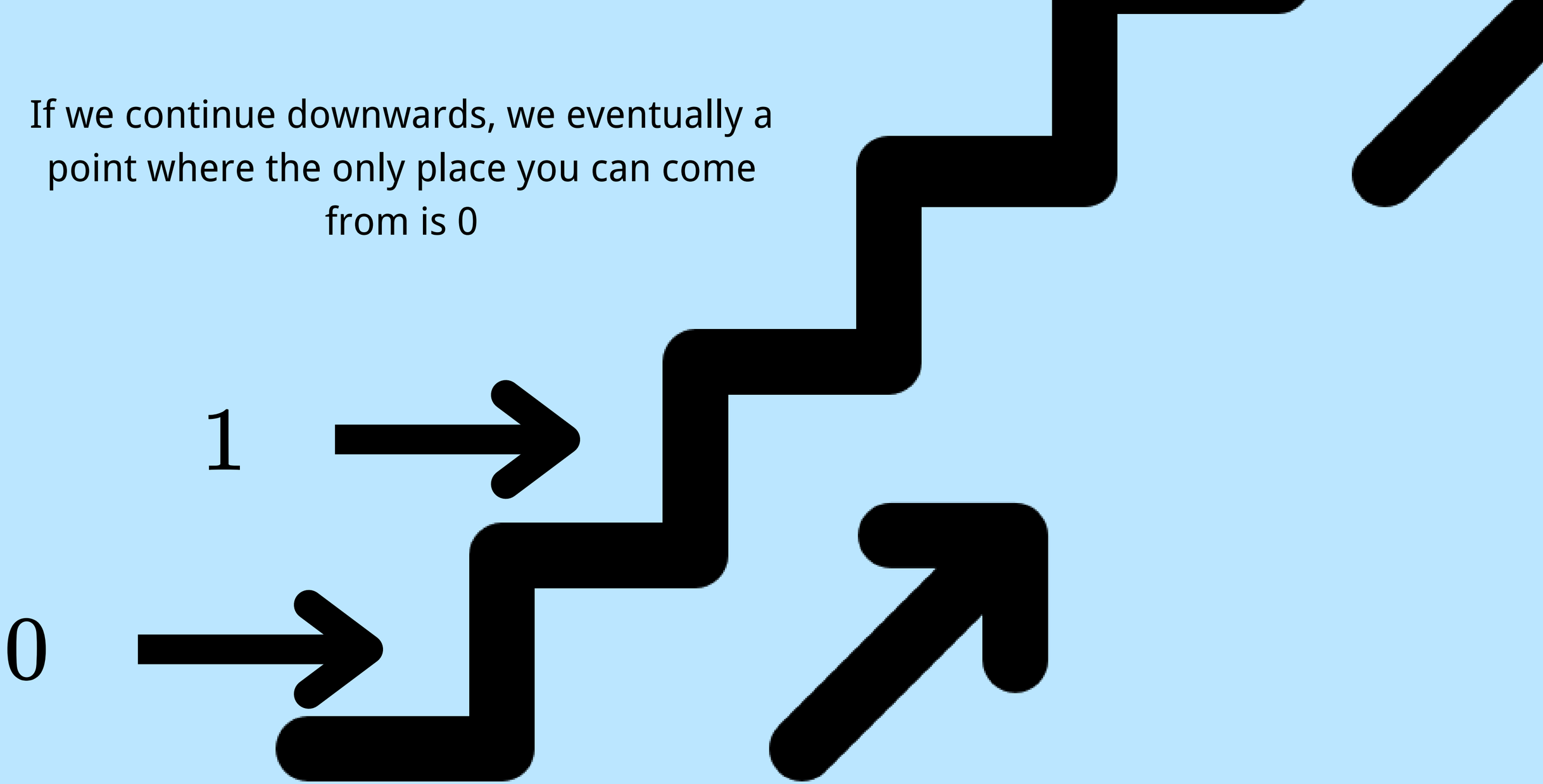
Consider this position



These are the only places you can come from



If we continue downwards, we eventually a point where the only place you can come from is 0



Let f(stair case height) = number of ways to get there. We know that

$$f(1) = 1 \quad f(0) = 1$$

Since we can only move 1 or 2 steps above, the methods to reach i should be equal to the sum of the methods to reach i-1 and i-2

$$f(i) = f(i-1) + f(i-2)$$

118. Pascal's Triangle

Easy | Topics | Companies

Given an integer `numRows`, return the first numRows of **Pascal's triangle**.

In **Pascal's triangle**, each number is the sum of the two numbers directly above it as shown:

Example 1:

```
Input: numRows = 5
Output: [[1],[1,1],[1,2,1],[1,3,3,1],[1,4,6,4,1]]
```

Example 2:

```
Input: numRows = 1
Output: [[1]]
```

Constraints:

- `1 <= numRows <= 30`

If we store each layer as a list and observe its indices.

Let the list be a[]

$$a[0], a[1], a[2], a[3], a[4], a[5], \ldots, a[n]$$

$$a[0], a[0] + a[1], a[1] + a[2] \ldots, a[n-1] + a[n]$$

This allows:

$$\begin{array}{r} 1, 2, 1 \\ 1, 2, 1 \\ \hline 1, 3, 3, 1 \end{array}$$

This means we can express layer i using layer i-1

Let f(index of the layer) = the layer’s array

```python
def f(i):
    if i == 0:
        return [1]
    else:
        previous_layer = f(i-1)
        copy = previous_layer.copy()
        copy.append(0)
        previous_layer.insert(0, 0)
        new_layer = []
        for j in range(len(copy)):
            new_layer.append(copy[j] + previous_layer[j])
        return new_layer
```

338. Counting Bits

Easy | Topics | Companies | Hint

Given an integer `n`, return *an array* `ans` *of length* `n + 1` *such that for each* `i` (`0 <= i <= n`), `ans[i]` *is the* ***number of*** `1`***'s*** *in the binary representation of* `i`.

Example 1:

```
Input: n = 2
Output: [0,1,1]
Explanation:
0 --> 0
1 --> 1
2 --> 10
```

Example 2:

```
Input: n = 5
Output: [0,1,1,2,1,2]
Explanation:
0 --> 0
1 --> 1
2 --> 10
3 --> 11
4 --> 100
5 --> 101
```

Constraints:

- $0 <= n <= 10^5$

Follow up:

- It is very easy to come up with a solution with a runtime of `O(n log n)`. Can you do it in linear time `O(n)` and possibly in a single pass?
- Can you do it without using any built-in function (i.e., like `__builtin_popcount` in C++)?

For odd number, it only has one more ‘1’ at the end of its binary representation

For even number, it would be the binary representation of its half with a ‘0’ at the end

$$f(i) = \begin{cases} f(i/2) & \text{i is even} \\ f(i-1)+1 & \text{i is odd} \end{cases}$$

```python
def f(n):
    if n == 0:
        return 0
    else:
        if n %2 == 0:
            return f(n//2)
        else:
            return 1 + f(n//2)
```